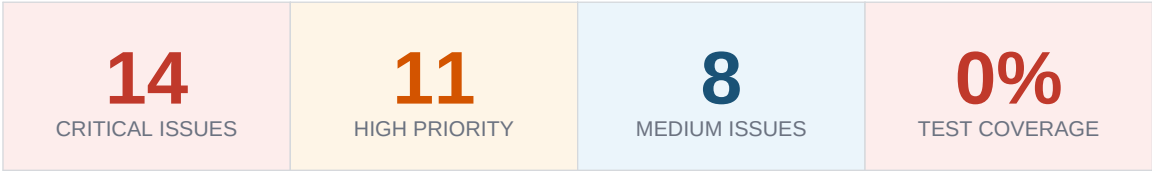


SMART KID MEALS

Play Store Compliance, Security & Production Readiness Audit

Audit Date: May 01, 2026 | Repository: github.com/nandhujayan/smart-kid-meals | Version: 0.0.0



FINAL VERDICT: CANNOT BE APPROVED FOR PLAY STORE

This is a Vite/React SPA — not an Android app. It contains a production backdoor, a mocked OTP auth system (password: 12345), no Privacy Policy, no account deletion, no Play Billing integration, and collects children's health data without parental consent. Estimated time to submission readiness: 6–8 weeks.

TABLE OF CONTENTS

- 1. Executive Summary & App Architecture
- 2. Play Store Compliance & Rejection Risk Analysis
- 3. Critical Issues — Fix Immediately (14 items)
- 4. High Priority Issues (11 items)
- 5. Security & Privacy Audit
- 6. Screen-by-Screen Audit
- 7. Architecture & Scalability
- 8. UX / UI Issues
- 9. Edge Cases & Operational Risks
- 10. Remediation Roadmap

1. EXECUTIVE SUMMARY & APP ARCHITECTURE

App Overview

App Name	Mom's Kitchen / Smart Kid Meals
Repository	github.com/nandhujayan/smart-kid-meals
Tech Stack	Vite 5 + React 18 + TypeScript + Tailwind CSS + shadcn/ui
Backend	Supabase (Auth, PostgreSQL, Edge Functions via Deno)
AI Provider	Google Gemini 2.0 Flash via Supabase Edge Functions
App Type	Web SPA — NOT a native Android app
Version	0.0.0 (scaffold default — never updated)
Package Name	vite_react_shadcn_ts (scaffold default — Play Store ID not set)
Test Coverage	~0% — only test is expect(true).toBe(true)
Audit Date	May 01, 2026

What the App Does

Smart Kid Meals is an AI-powered meal suggestion and nutrition tracking app for parents (primarily mothers). Using Google Gemini AI, it generates age-appropriate meal recipes for children based on dietary preferences, allergies, health goals, and cuisine. It also provides weekly meal planning, water/hydration tracking, liquid nutrition suggestions, child growth analytics, and a grocery list generator.

What the App Does NOT Have: No delivery system. No marketplace. No mom-to-user meal ordering. The concept brief mentions 'Meals prepared by Moms' — this is a positioning statement, not an implemented feature. The actual app is an AI recipe generator.

2. PLAY STORE COMPLIANCE & REJECTION RISK ANALYSIS

Issue	Policy	Risk
Not a native Android app — no APK/AAB exists	Minimum Functionality	BLOCKER
No Privacy Policy linked in app or store listing	Privacy, Security §4.1	HIGH
No account deletion mechanism	Account Deletion Requirement §4.22	HIGH
Children's data collected without parental consent	Families Policy + COPPA	HIGH
Phone OTP is mocked (OTP = '12345')	Deceptive Behavior §4.8	HIGH
Hardcoded phone number grants free Pro tier	Deceptive Behavior §4.8	HIGH
Subscription payment button is disabled/non-functional	Payments Policy §Billing	HIGH
No Google Play Billing for in-app purchases	Payments Policy §Billing	HIGH
GitHub OAuth offered in consumer children's app	Families Policy	MEDIUM
AI nutrition claims without medical disclaimer	Health §Health features	MEDIUM
Data Safety section would be inaccurate/incomplete	Data Safety Form	MEDIUM
DiceBear API receives child names (3rd-party data leak)	Families Policy	MEDIUM
App name 'Mom's Kitchen' ≠ repo 'smart-kid-meals'	Spam & Misleading §Misleading	LOW

3. CRITICAL ISSUES — FIX IMMEDIATELY

C-01 NOT AN ANDROID APP

BLOCKER

The entire codebase is a Vite/React SPA. There is no AndroidManifest.xml, no Gradle build, no APK or AAB output. The Play Store requires an Android app bundle. This must be wrapped using Capacitor.js or Trusted Web Activity (TWA), or rebuilt natively, before any Play Store submission is even possible. Every other issue below applies to that wrapper app too.

C-02 PRODUCTION BACKDOOR — Hardcoded phone grants Pro access

SECURITY

In useAuth.tsx, a specific Indian mobile number (7012793080) is hardcoded to automatically grant Pro subscription status. Anyone who discovers this number gets lifetime Pro access for free. This is a developer backdoor left in production code.

```
// useAuth.tsx line 36 if (phone === '7012793080' || phone === '+917012793080') {
  setSubscriptionTier('pro'); // production backdoor }
```

C-03 MOCK OTP AUTHENTICATION — OTP is literally '12345'

AUTH FAILURE

The phone authentication tab sends no real SMS. The OTP verification accepts the hardcoded value '12345' for any account. The UI even displays the hint to the user. In production, any user's account can be accessed by entering '12345'. This is not authentication — it is a placeholder.

```
// AuthModal.tsx if (otp === '12345') { signInAsGuest(); } // anyone can log in as anyone
```

C-04 NO PRIVACY POLICY

PLAY POLICY

Google Play mandates a Privacy Policy URL for all apps, and it is mandatory for apps collecting personal data. This app collects: email, phone number, child names, ages, weights, heights, dietary preferences, health goals, and AI generation history. No privacy policy exists anywhere in the codebase.

C-05 CHILDREN'S DATA WITHOUT PARENTAL CONSENT — COPPA / DPDP Violation

LEGAL RISK

The app collects and stores health data about minors (children) including names, ages, weights, heights, and health goals in a Supabase database. Under the US COPPA Act and India's Digital Personal Data Protection (DPDP) Act 2023, verifiable parental consent must be obtained before collecting any data related to a child under 13/18. No consent screen, age gate, or parental verification exists.

C-06 NO ACCOUNT DELETION MECHANISM

PLAY POLICY

Since August 2022, Google Play requires all apps with account creation to provide an in-app account deletion option. The app has a sign-out button that calls supabase.auth.signOut() but this does NOT delete user data. No 'Delete Account' option exists in any menu or settings screen.

C-07 PAYMENT BUTTON IS DISABLED — No Google Play Billing

PLAY POLICY

The Pricing modal shows a '\$4.99/month Go Premium' button that is permanently disabled. No payment SDK is integrated. Android apps selling digital subscriptions MUST use Google Play Billing API. Third-party payment processors (Stripe, Razorpay) are prohibited for digital goods on Android.

C-08 SchemaType IS UNDEFINED — Runtime crash risk

CODE BUG

In gemini.ts, SchemaType.OBJECT, SchemaType.STRING, SchemaType.NUMBER etc. are used to define mealSchema, weeklyPlanSchema, and drinkSchema — but SchemaType is never imported. These code paths will throw ReferenceError: SchemaType is not defined at runtime. The schemas are also never passed to the edge function, making them entirely dead code.

C-09 VITE_ PREFIX EXPOSES GEMINI KEY IN CLIENT BUNDLE

SECURITY

Environment variables prefixed with VITE_ are compiled into the client-side JavaScript bundle by Vite. VITE_GEMINI_API_KEY referenced in diagnostics.ts will be visible in the production JS bundle to anyone who opens browser DevTools. The edge function architecture correctly hides the server-side key, but the client-side diagnostic tool still reads and uses the exposed key.

C-10 NO MEDICAL DISCLAIMER — AI Health Claims

LEGAL RISK

The app generates calorie counts, macronutrient ratios, and health recommendations for children via AI. These are presented as authoritative facts with no disclaimer. Play Store Health policy prohibits misleading health claims. A clear disclaimer must be shown throughout: this is not a medical device and does not replace professional dietary advice.

C-11 console.log() OUTPUTS CHILD DATA IN PRODUCTION

PRIVACY

MealResult.tsx line 22 logs the full meal object (including child profile names) on every render. useAuth.tsx logs the backdoor phone number when activated. Production builds must strip all console.log statements containing personal or child data.

C-12 OPEN CORS IN EDGE FUNCTION — Any Origin Allowed

SECURITY

The Supabase edge function sets 'Access-Control-Allow-Origin: *' allowing requests from any domain. While JWT auth validation is in place, wildcard CORS enables cross-site script inclusion attacks. Restrict to your production domain only.

C-13 DATA SAFETY FORM WOULD BE INACCURATE

PLAY POLICY

The Play Store Data Safety form requires declaring all personal data collected. Misrepresentation is a policy violation triggering removal even post-approval. Child health data is a sensitive data type requiring explicit disclosure. The form must accurately declare: name, email, phone, child health metrics, dietary data, and usage analytics.

C-14 ZERO MEANINGFUL TEST COVERAGE

QUALITY

The only test is: expect(true).toBe(true). No tests exist for authentication flows, meal generation, subscription checks, data migration, or any business logic. For an app storing children's health data at scale, this is not acceptable for production deployment.

4. HIGH PRIORITY ISSUES

H-01 Guest user bypass of rate limiting

RATE LIMIT

Guest users are mock objects not stored in Supabase. The `incrementUsage()` function skips non-UUID IDs. Guests have unlimited local AI mock generations with no tracking.

H-02 No onboarding flow

UX

New users land directly on the meal generation form with no tutorial, no value explanation, and no profile setup prompt. This critically harms first-session retention and Day-1 activation.

H-03 Notification button is cosmetic — no real notifications

BROKEN FEATURE

The bell icon only writes to `localStorage`. No service worker, no push subscription, no notification scheduling exists. Users expect real meal reminders.

H-04 Child's name sent to DiceBear API (3rd-party)

PRIVACY

Avatar URL is `https://api.dicebear.com/7.x/avataaars/svg?seed={childName}`. The child's name is transmitted to an external server. Under DPDP/COPPA this is an unauthorized data disclosure.

H-05 `window.location.reload()` on sign-out

CODE QUALITY

Sign-out triggers a full page reload to 'clear React state.' This is a code smell, breaks back navigation in `WebView`, and creates jarring UX. Use proper React context reset.

H-06 Weekly plan generates duplicate IDs

BUG

In `gemini.ts`, weekly plan IDs use `Date.now()` in a synchronous loop — all 7 days get the same timestamp. Use `crypto.randomUUID()` instead.

H-07 `localStorage` as primary data layer — no quota handling

DATA RISK

Guest users store meals, profiles, and plans in `localStorage` (5–10 MB limit). No quota check. Silent data loss when limit is exceeded. No warning shown before clearing browser data.

H-08 No offline support / error on network loss

RELIABILITY

No service worker. No timeout on Supabase function calls. Loading spinner hangs indefinitely on connection loss. Critical for rural Indian users with intermittent connectivity.

H-09 System Status debug panel visible to end users

UX

The `SystemStatus` component is exposed in the user-facing sidebar. It references 'Vercel', exposes API connection details, and has a 'Run Test' button. Remove from production UI entirely.

H-10 AI meal cache never expires

DATA QUALITY

The `ai_meal_cache` table has no TTL. A cached meal from launch day will be served indefinitely. Nutrition science evolves; implement a 30–90 day soft expiry.

H-11 `@google/generative-ai` bundled in frontend

PERFORMANCE

The 4+ MB Google AI SDK is a production dependency imported in `diagnostics.ts`. Since AI generation is server-side (edge functions), this entire package can be removed, significantly reducing bundle size.

5. SECURITY & PRIVACY AUDIT

Authentication

- Phone OTP is mock (OTP=12345) — any account can be taken over
- Hardcoded phone number grants Pro subscription bypass
- GitHub OAuth inappropriate for children's consumer app
- No password strength requirements (minimum length, complexity)
- No session timeout — shared family device risk
- No re-authentication before sensitive actions (delete profile, cancel subscription)

Data Protection

- Child health data (weight, height, age) stored as plain text — no field-level encryption
- Child names transmitted to third-party DiceBear API without consent
- localStorage contains unencrypted child profile data
- console.log leaks meal data and child profile names in production
- No data retention policy — child data stored indefinitely

API Security

- Edge function allows wildcard CORS (Access-Control-Allow-Origin: *)
- VITE_GEMINI_API_KEY exposed in client bundle via Vite env prefix
- No input sanitization on allergy/ingredient tag fields (prompt injection risk)
- AI cache hash is deterministic — user dietary data influences other users' results

Compliance

- No COPPA compliance — no parental consent flow for child data
- No DPDP Act 2023 compliance — no consent management for Indian users
- No GDPR compliance — no data export, no right to erasure
- No data processing agreement disclosed for Supabase and Google Gemini

6. SCREEN-BY-SCREEN AUDIT

Auth Modal (Login/Register)

Issues Found	Required Fixes
• Mock OTP accepts '12345' for any number • GitHub OAuth exposed (not suitable for consumer/family app) • No 'Forgot Password' link • No privacy policy link before account creation	• Wire real Supabase phone OTP • Remove GitHub OAuth provider • Add forgot-password flow • Add privacy policy + ToS links before signup

Meal Form (Generate Tab)

Issues Found	Required Fixes
• Form state saved to localStorage unencrypted • No character limit on ingredient tags (XSS risk) • Weekly mode shares same form — UI is confusing	• Sanitize all text inputs before storage • Cap allergy/ingredient tags at 15 items • Create visually distinct weekly vs single mode

Meal Result (AI Output Screen)

Issues Found	Required Fixes
• console.log(meal) fires on every render • No medical disclaimer • No allergen cross-check alert	• Remove console.log • Add 'Not medical advice' disclaimer • Highlight if AI output contains declared allergen

Weekly Planner (7-Day View)

Issues Found	Required Fixes
• Weekly generation uses full daily AI quota at once • No single-meal regeneration within plan	• Show usage cost warning before generation • Allow per-meal regeneration in weekly plan

Child Profiles (Profile Management)

Issues Found	Required Fixes
• Child name sent to DiceBear third-party API • Profile count limit (2 free) not enforced server-side • No data deletion confirmation dialog	• Generate avatars locally (initials/color) • Enforce profile limits in database via RLS • Add confirmation before profile deletion

Growth Analytics (Growth Tracking)

Issues Found	Required Fixes
• No WHO growth chart reference lines • No unit labels (kg/lb, cm/in) • No abnormal growth alerts	• Add WHO percentile reference bands • Add unit toggle (metric/imperial) • Add 'Consult pediatrician' alert for abnormal values

Pricing Modal (Subscription Screen)

Issues Found	Required Fixes
• Go Premium button is disabled — does nothing • USD pricing for Indian market • No Google Play Billing integration	• Integrate Google Play Billing API • Show INR pricing with regional detection • Add free trial option

System Status (Debug Panel)

Issues Found	Required Fixes
• Should NOT be visible to end users • References internal infra (Vercel) • Exposes API connection status to all users	• Remove from user-facing app entirely • Move to admin-only developer panel

7. ARCHITECTURE & SCALABILITY

What is done well

- Supabase Edge Function protects Gemini API key server-side — correct pattern
- SHA-256 input hash caching reduces AI API costs significantly
- Row Level Security (RLS) enabled on all database tables
- `handle_new_user` trigger auto-creates subscription and usage rows
- `increment_usage_v2` function implements daily reset logic correctly
- Local-first with cloud sync architecture is appropriate for the use case
- Daily generation limits enforced server-side (not just client-side)

Architecture issues to fix

Cache never expires: `ai_meal_cache` has no TTL. Implement 30–90 day soft expiry with background regeneration.

No error boundaries: Any component crash (malformed AI JSON, null profile) unmounts the entire React tree. Wrap `MealResult`, `WeeklyPlanner`, and `GrowthAnalytics` in React Error Boundaries.

No offline support: No service worker or optimistic updates. Add a timeout (30s) on edge function calls with user feedback.

Missing RLS policies: `api_usage_logs` and `usage_stats` lack explicit client-accessible policies, causing silent failures for client-side reads.

Bundle size: `@google/generative-ai` (4+ MB) is a frontend dependency used only for a diagnostics tool. Remove it — the edge function handles all AI calls.

ID collision in weekly plan: `Date.now()` IDs in a synchronous loop produce identical IDs for all 7 days. Use `crypto.randomUUID()`.

8. UX / UI ISSUES	
No onboarding flow 3–5 step onboarding to capture first child profile and explain AI meal generation.	HIGH
No empty states Bare text when no saved meals/profiles exist. Each empty state needs illustration + CTA button.	MEDIUM
Accessibility failures (WCAG 2.1 AA) Missing ARIA labels on buttons, no focus rings visible on chip selects, loading overlay has no live region, color-only status indicators.	HIGH
Notification bell is cosmetic Remove or implement via Web Push API. Never ship broken promises to users.	HIGH
App name inconsistency UI shows 'Mom's Kitchen Kids'. Repo is 'smart-kid-meals'. Play Store listing, app ID, and branding must be identical.	MEDIUM
ThemePicker is dead code ThemePicker component exists but is rendered nowhere in the app navigation.	LOW
Hard reload on sign-out window.location.reload() breaks WebView back navigation. Reset state via React context.	MEDIUM
GitHub login in auth modal Irrelevant for a parent/child cooking app. Confuses users. Remove it.	HIGH
USD pricing for Indian market App is clearly India-focused (phone number hint uses +91, Halal/Indian diet options). Show INR by default.	HIGH
System Status in end-user UI Developer debug panel must not be visible to real users. Remove entirely from user-facing navigation.	HIGH

9. EDGE CASES & OPERATIONAL RISKS

AI returns malformed JSON: No try/catch around JSON.parse in the client-side weekly plan processor. App crashes silently. Add validation and fallback to cached meals.

User clears browser storage (guest): All guest data is permanently lost. No warning. Show 'Sign up to save your data' prompt before data is lost.

Rate limit hit mid-weekly-plan: If the daily limit is hit during weekly plan generation (which uses one bulk call), the user gets a confusing error after waiting 30+ seconds. Show estimated usage before generation.

Concurrent saves from two devices: If a user has the app open on two devices and saves a meal simultaneously, the later write wins silently. Implement optimistic locking or last-write-wins with user notification.

Child profile deleted mid-session: If a profile is deleted from another session while the current session has it active, the activeProfile state becomes stale. Add profile existence validation on resume.

Gemini API outage: The fallback to cached meals by age/mealType is good, but if the cache is empty for a new age group, the app throws the original AI error. Ensure the mock data library is always the final fallback.

Spoiled food / wrong allergy output: AI generates meal with an allergen the user declared. No cross-check validation after AI response. Implement a post-generation allergen scanner that flags mismatches before showing results.

User in unsupported Gemini region: diagnostics.ts handles 'User location is not supported' from Gemini. The edge function does not. Add regional Gemini endpoint handling in the edge function.

10. REMEDIATION ROADMAP

Week 1–2

- Wrap app in Capacitor.js or build native Android shell
- Remove hardcoded phone number backdoor from useAuth.tsx
- Wire real Supabase phone OTP (disable mock '12345' entirely)
- Remove GitHub OAuth from AuthModal
- Fix SchemaType import error in gemini.ts
- Strip all console.log from production build
- Remove @google/generative-ai from frontend dependencies
- Restrict CORS in edge function to production domain

Week 3–4

- Write and publish Privacy Policy (see separate document)
- Write and publish Terms & Conditions (see separate document)
- Build in-app Account Deletion flow with data purge
- Add Parental Consent screen before any child data is collected
- Replace DiceBear with local avatar generation
- Add medical disclaimer throughout app
- Add allergen cross-check validation on AI output
- Fix Data Safety form to accurately declare all data collected

Week 5–6

- Integrate Google Play Billing API for Pro subscription
- Convert USD pricing to INR for Indian market
- Implement proper React Error Boundaries on all major sections
- Add 30s timeout + user feedback for edge function calls
- Implement session timeout (auto-lock after 30 minutes)
- Add cache TTL (90 days) to ai_meal_cache table
- Fix weekly plan ID collision (use crypto.randomUUID())
- Add explicit RLS policies for usage_stats and api_usage_logs

Week 7–8

- Build 3–5 step onboarding flow with first child profile capture
- Design and implement all empty states with CTAs
- Accessibility audit: ARIA labels, focus rings, live regions
- Implement real Web Push notifications (or remove button)
- Remove System Status from user-facing UI
- Write real test suite (target 60%+ coverage on critical paths)
- Beta test with 50 real families — collect feedback

■ Complete Play Store store listing with accurate description and screenshots

Report generated: May 01, 2026 | Audited by: Senior Play Store Compliance & Production Readiness Review | This report is confidential and intended for the development team only.